



Desarrollo Backend III (PBY2203)

Informe Técnico · Evaluación Final Transversal

Desarrollo Backend Avanzado: Spring Cloud & Batch

Felipe Peñaloza Oyarzún — DUOC UC

Índice

Actualiza el índice en Word: clic derecho → Actualizar campos (F9).

1. Resumen ejecutivo

El Banco XYZ es una institución con más de 30 años de operación cuyo núcleo transaccional reside en una plataforma legacy basada en COBOL y scripts Shell sobre mainframe. Este informe documenta la modernización de ese sistema hacia una arquitectura de microservicios en la nube, resiliente y segura, desarrollada con Spring Cloud y Spring Batch.

La solución cubre las tres capacidades centrales del dominio bancario: la migración de los procesos batch legacy, la implementación del patrón Backend for Frontend (BFF) para los canales web, móvil y cajero, y la creación de microservicios de negocio (Cuentas, Pagos y Clientes) con seguridad distribuida OAuth 2.0, tolerancia a fallos con Resilience4j y mensajería asíncrona con Apache Kafka. Todo el ecosistema se empaqueta en contenedores Docker y se orquesta con Docker Compose, preparado para desplegarse y escalar horizontalmente en AWS.

2. Procesos clave identificados para la migración

Se identificaron cinco procesos clave que estructuran la migración desde el monolito legacy hacia la arquitectura de microservicios:

- Migración de procesos batch a Spring Batch: reescritura de los reportes y cálculos masivos (transacciones diarias, intereses y estados de cuenta) con lectura, procesamiento, validación y escritura por lotes.
- División del sistema monolítico en microservicios: separación por dominio de negocio en servicios independientes (Cuentas, Pagos, Clientes), desplegables y escalables por separado.
- Implementación del patrón Backend for Frontend (BFF): una capa por canal (web, móvil, cajero) que optimiza las respuestas y aplica seguridad específica.
- Seguridad distribuida con Spring Cloud Security: autorización centralizada mediante un Authorization Server OAuth 2.0 y microservicios como Resource Servers.

- Integración de mensajería asíncrona con Apache Kafka: arquitectura orientada a eventos que desacopla los servicios mediante publicación/suscripción.

3. Propuesta de arquitectura

La arquitectura propuesta responde a los requerimientos clave del negocio identificados en el caso. A continuación se justifica cada decisión:

Requerimiento de negocio	Decisión arquitectónica	Justificación
Escalabilidad ante demanda variable	Microservicios + escalado horizontal (réplicas, balanceo por Eureka/Gateway)	Permite escalar solo el servicio con carga, optimizando recursos y costos frente al escalado vertical del monolito.
Tolerancia a fallos y alta disponibilidad	Resilience4j (Circuit Breaker + fallback)	Aísla los fallos: la caída de un servicio no se propaga al resto; el sistema degrada de forma controlada.
Seguridad en entorno distribuido	OAuth 2.0 con Authorization Server central + Resource Servers	Centraliza la autorización y evita duplicar credenciales y lógica de seguridad en cada servicio (delegación de seguridad).

El siguiente diagrama presenta la arquitectura general del sistema, desde los canales de cliente hasta la capa de datos, pasando por el BFF, el API Gateway, los microservicios de negocio y la infraestructura Spring Cloud.

Arquitectura de microservicios — Banco XYZ

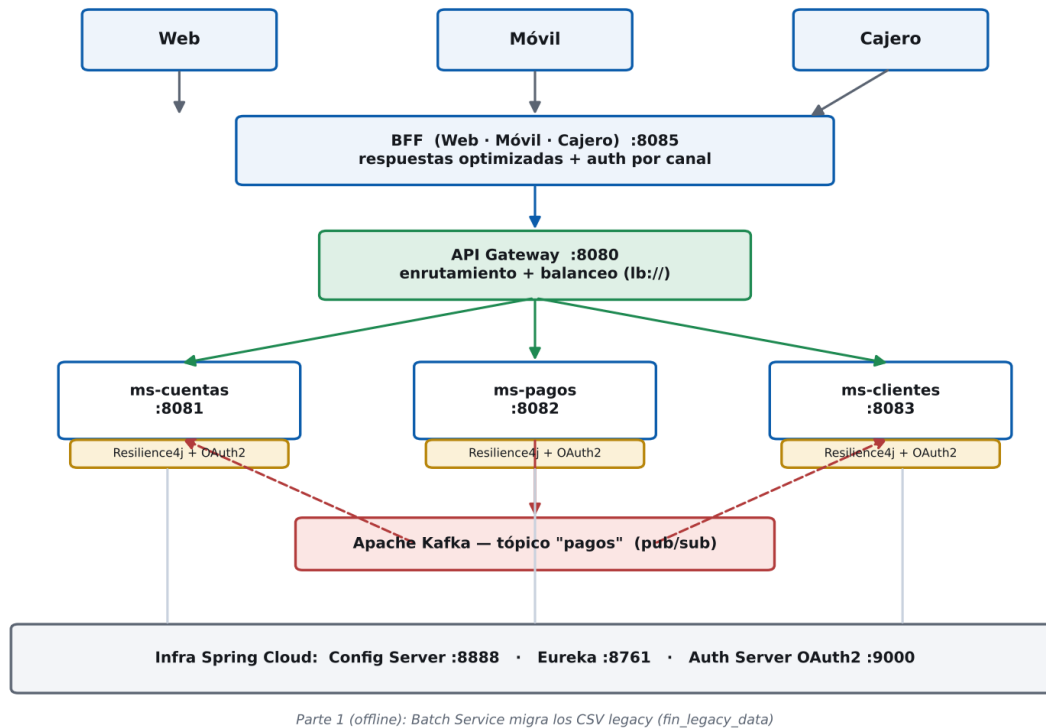


Figura 1. Arquitectura de microservicios del Banco XYZ.

4. Parte 1 — Migración de procesos batch con Spring Batch

Los tres procesos batch legacy se reescribieron como Jobs de Spring Batch, tomando como base el conjunto de datos oficial fin_legacy_data. Cada Job sigue el patrón leer–procesar–escribir, con validación de anomalías, políticas de omisión (skip) y reintento, listeners de registro y procesamiento en paralelo mediante particionamiento.

Parte 1 — Migración batch con Spring Batch

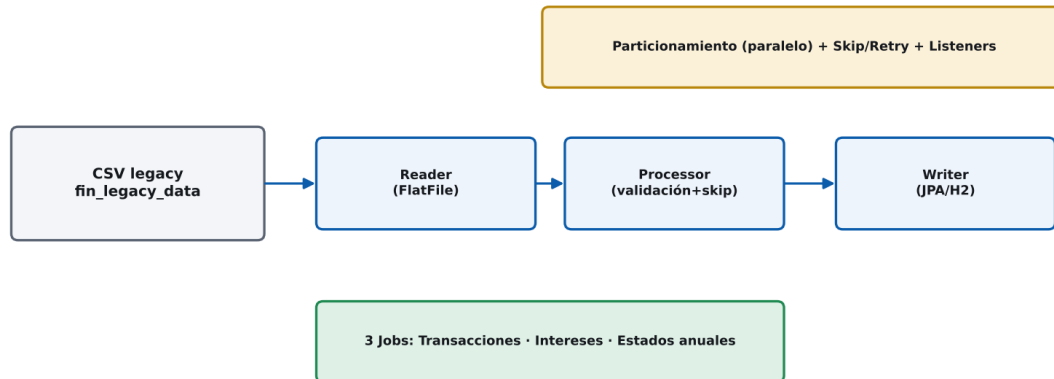


Figura 2. Flujo del procesamiento batch (reader → processor → writer).

El dataset incluye anomalías intencionales (montos negativos, fechas mal formateadas, tipos inválidos, edades fuera de rango, duplicados y descripciones vacías). Los resultados obtenidos sobre el conjunto de volumen (1.000 registros por archivo) fueron:

Job (proceso)	Registros válidos	Omitidos (skip)	Principales anomalías
1 · Reporte de Transacciones Diarias	≈ 392	≈ 608	tipo inválido, monto ≤ 0, fecha mal formada
2 · Cálculo de Intereses	≈ 340	≈ 660	edad fuera de rango, saldo faltante, tipo inválido
3 · Estados de Cuenta Anuales	≈ 952	≈ 48	monto no numérico (se conservan negativos como retiros)

Manejo avanzado de errores, paralelismo y escalabilidad:

- Cada anomalía lanza una excepción controlada que la SkipPolicy omite y el SkipListener registra (log + archivo de errores), sin detener el Job.
- El particionamiento divide cada archivo en N particiones ejecutadas en paralelo (hilos Batch-Part-1/2/3), configurable para comparar rendimiento.
- Cada ejecución agrega un parámetro único (timestamp), permitiendo la reejecución automática ante fallos críticos.

 **ESPACIO PARA EVIDENCIA — Consola con los 3 Jobs en estado COMPLETED, los hilos de paralelismo [Batch-Part-N] y el resumen de leídos/escritos/omitidos.**

[Inserta aquí tu captura de pantalla]

 **ESPACIO PARA EVIDENCIA — Consola H2 con las tablas pobladas (TRANSACCION, CUENTA_INTERES, ESTADO_CUENTA_ANUAL).**

[Inserta aquí tu captura de pantalla]

5. Parte 2 — Patrón Backend for Frontend (BFF)

Se implementó el patrón BFF con una capa de backend independiente por canal, cada una con respuestas optimizadas y seguridad propia por rol. Las credenciales de un canal no sirven en otro, y la operación crítica de retiro queda restringida al canal de cajero.

Canal	Prefijo	Rol	Optimización de la respuesta
Web	/api/web	WEB	Datos completos: dashboard, tarjetas e historial para interfaces ricas.
Móvil	/api/movil	MOVIL	Respuestas ligeras: solo datos esenciales, para ahorrar ancho de banda.
Cajero	/api/cajero	CAJERO	Mínima y segura: consulta de saldo y retiro.

 **ESPACIO PARA EVIDENCIA — Las tres respuestas de la MISMA cuenta por cada canal (web completa vs móvil ligera vs cajero mínima) y un 403 al usar credenciales cruzadas.**

[Inserta aquí tu captura de pantalla]

6. Parte 3 — Microservicios resilientes y seguros

El monolito se dividió en tres microservicios de negocio independientes, cada uno con su propia base de datos y ciclo de vida: Gestión de Cuentas (ms-cuentas), Procesamiento de Pagos (ms-pagos) y Gestión de Clientes (ms-clientes). Se apoyan en la infraestructura Spring Cloud: Config Server (configuración centralizada), Eureka (descubrimiento) y API Gateway (enrutamiento y balanceo).

6.1 Seguridad distribuida con OAuth 2.0

Un Authorization Server (Spring Authorization Server) emite access tokens JWT firmados con RSA y publica su JWK Set. Cada microservicio actúa como Resource Server y valida los tokens contra ese JWK, delegando la autenticación y evitando gestionar credenciales

localmente. Se utiliza el flujo client_credentials para la comunicación entre servicios y clientes de confianza.

6.2 Tolerancia a fallos con Resilience4j

Los endpoints de negocio están protegidos con Circuit Breaker y métodos de fallback: ante un fallo en la capa de datos o en un servicio dependiente, el sistema responde con una respuesta de respaldo en lugar de propagar el error, manteniendo la disponibilidad.

6.3 Mensajería asíncrona con Apache Kafka

El microservicio de Pagos publica un evento en el tópico 'pagos' cada vez que se registra un pago. Los microservicios de Cuentas y Clientes lo consumen de forma independiente (con distinto groupId): Cuentas actualiza el saldo y Clientes notifica al titular. Esto desacopla los servicios y habilita casos como notificaciones y alertas de seguridad.

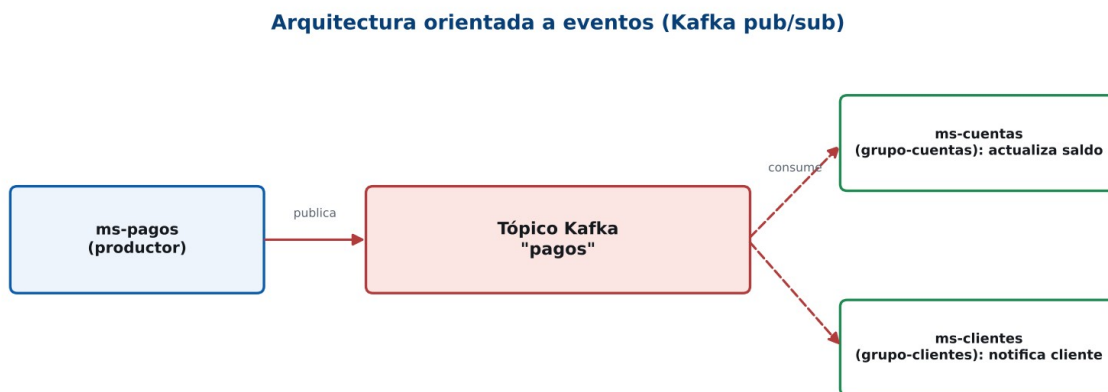


Figura 3. Arquitectura orientada a eventos (publish/subscribe sobre Kafka).

 **ESPACIO PARA EVIDENCIA** — Token OAuth2 obtenido (200) y una llamada protegida con token (200) frente a una sin token (401).

[Inserta aquí tu captura de pantalla]

 **ESPACIO PARA EVIDENCIA** — Logs de ms-cuentas (saldo actualizado) y ms-clientes (notificación) tras publicar un pago, y el tópico 'pagos' en Kafka-UI.

[Inserta aquí tu captura de pantalla]

 **ESPACIO PARA EVIDENCIA** — Estado del Circuit Breaker (actuador/circuitbreakers) demostrando la tolerancia a fallos.

[Inserta aquí tu captura de pantalla]

7. Contenerización y despliegue en la nube

Cada componente cuenta con un Dockerfile multi-stage (compilación con Maven y una imagen final liviana con JRE). El archivo docker-compose.yml orquesta todo el ecosistema — infraestructura, microservicios y Kafka— en una red común. Los microservicios de negocio están preparados para escalado horizontal: al levantar réplicas, cada instancia se registra en Eureka y el API Gateway reparte la carga entre ellas.

Comando de escalado horizontal:

```
docker compose up -d --scale ms-pagos=3 --scale ms-cuentas=2 --scale ms-clientes=2
```

El despliegue en AWS se realiza sobre una instancia EC2 con Docker, exponiendo los puertos necesarios mediante el Security Group; el detalle paso a paso está en el documento [despliegue.md](#).

 **ESPACIO PARA EVIDENCIA** — docker compose ps mostrando todos los servicios y las réplicas por microservicio (escalabilidad horizontal).

[Inserta aquí tu captura de pantalla]

 **ESPACIO PARA EVIDENCIA** — Dashboard de Eureka con los servicios y sus múltiples instancias registradas.

[Inserta aquí tu captura de pantalla]

 **ESPACIO PARA EVIDENCIA** — (Despliegue AWS) Consola EC2 con la instancia en ejecución y docker ps dentro de la instancia.

[Inserta aquí tu captura de pantalla]

8. Comparación con el sistema legacy

Aspecto	Sistema legacy (COBOL/mainframe)	Nuevo sistema (Spring Cloud)
Escalabilidad	Vertical, costosa	Horizontal, por servicio
Tolerancia a fallos	Un fallo afecta a todo el sistema	Circuit breakers aíslan los fallos
Integración	Rígida y acoplada	APIs REST + eventos asíncronos (Kafka)
Seguridad	Centralizada y limitada	OAuth 2.0 distribuida por servicio
Procesos batch	Scripts Shell	Spring Batch con reintentos y paralelismo
Despliegue	Manual, monolítico	Contenedores orquestados, listos para la nube

9. Desafíos enfrentados y soluciones

- Calidad de los datos legacy: el dataset traía anomalías intencionales. Se resolvió con validación por processor + SkipPolicy y SkipListener, garantizando integridad sin perder la traza de los errores.
- Seguridad entre servicios sin acoplar credenciales: se migró del JWT propio a un Authorization Server OAuth 2.0 con Resource Servers, centralizando la autorización.
- Arranque ordenado del ecosistema en contenedores: se añadieron healthchecks y depends_on, más reintentos del cliente de Config Server, para tolerar las carreras de arranque.
- Comunicación asíncrona y desacoplada: se adoptó Kafka (pub/sub) para que el productor no dependa de los consumidores y se puedan agregar nuevos sin modificar el origen.

10. Propuestas de mejora y próximos pasos

- Migrar la persistencia de H2 a un motor gestionado (Amazon RDS: MySQL/PostgreSQL).
- Gestionar secretos fuera del código con AWS Secrets Manager o variables de entorno.
- Orquestrar en producción con ECS/EKS en lugar de Docker Compose en una sola instancia.
- Incorporar observabilidad: métricas, trazas distribuidas y logs centralizados (p. ej. CloudWatch).
- Autorización fina por scope/rol en cada endpoint y validación del issuer del token.

11. Conclusión

La solución moderniza el núcleo del Banco XYZ en una arquitectura de microservicios segura, resiliente y escalable, cumpliendo los objetivos del caso: migración batch, patrón BFF por canal, microservicios con OAuth 2.0, Resilience4j y Kafka, y despliegue contenerizado

preparado para la nube. El resultado es un sistema más flexible, tolerante a fallos y adaptable que el monolito legacy, y una base sólida para evolucionar hacia un entorno productivo en la nube.

Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.